

6. Design of Image Intensity Inversion Circuit Using Xilinx Vivado

Aim

To design and simulate a Verilog-based image intensity inversion system that converts a 256×256 grayscale image into its negative image using a UUT module, with pixel data provided through a text file and the processed output written back to a text file.

Software Required

- Verilog HDL
- Xilinx Vivado
- Python (with Pillow and NumPy libraries) to convert image to text file and back
- Operating System: Windows

Procedure

1. A grayscale image is resized to 256×256 and converted into a one-dimensional text file containing 65,536 pixel values (0–255), with one pixel per line.
2. A Verilog UUT is designed to perform image intensity inversion, where each input pixel value is converted using the relation:
Output pixel = 255 – Input pixel.
3. A Verilog testbench is written to open the input text file, read pixel values sequentially, and apply them to the UUT.
4. The inverted output produced by the UUT for each pixel is captured by the testbench and written into an output text file.
5. The simulation is executed, and after completion, the generated output text file is converted back into an image using Python to visualize the intensity-inverted (negative) image.

Python program to convert the 256x256 image to 1D form suitable for verilog module

```
from PIL import Image

import numpy as np

img = Image.open("input_image.png").convert("L")

img = img.resize((256, 256))

img_array = np.array(img, dtype=np.uint8)

with open("image_in.txt", "w") as f:

    for pixel in img_array.flatten():

        f.write(str(int(pixel)) + "\n")
```

Testbench program

```

`timescale 1ns/1ps

module tb_intensity_inversion;

    reg    clk, rst;

    reg [7:0] pixel_in;

    reg    valid_in;

    wire [7:0] pixel_out;

    wire    valid_out;

    integer infile, outfile;

    integer r;

    intensity_inversion UUT (

        .clk(clk),

        .rst(rst),

        .pixel_in(pixel_in),

        .valid_in(valid_in),

        .pixel_out(pixel_out),

        .valid_out(valid_out)

    );

    // Clock (10 ns)

    always #5 clk = ~clk;

    initial begin

        clk = 0;

        rst = 1;

        valid_in = 0;

        pixel_in = 0;

        #20 rst = 0;

        infile = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_in.txt", "r");

        outfile = $fopen("I:\\Saveetha\\SOC\\Lab experiments\\image_out.txt", "w");

        if (infile == 0 || outfile == 0) begin

            $display("File open error");

            $finish;

        end

    end

```

```

while (!$feof(infile)) begin
    r = $fscanf(infile, "%d\n", pixel_in);
    valid_in = 1'b1;
    @(posedge clk); // input accepted
    @(posedge clk); // output available
    if (valid_out)
        $fdisplay(outfile, "%d", pixel_out);
end
valid_in = 0;
#20;
$fclose(infile);
$fclose(outfile);
$display("Intensity inversion finished.");
$finish;
end
endmodule

```

Verilog program for the image intensity inversion circuit

```

module intensity_inversion (
    input wire    clk,
    input wire    rst,
    input wire [7:0] pixel_in, // 0 to 255
    input wire    valid_in,
    output reg [7:0] pixel_out,
    output reg    valid_out
);
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pixel_out <= 8'd0;
        valid_out <= 1'b0;
    end
    else begin

```

```

if (valid_in) begin
    pixel_out <= 8'd255 - pixel_in; // Intensity inversion
    valid_out <= 1'b1;
end
else begin
    valid_out <= 1'b0;
end
end
end
endmodule

```

Python program to convert the 1D output data generated by verilog module to 256x256 image

```

import numpy as np
from PIL import Image
data = np.loadtxt("image_out.txt", dtype=np.uint8)
img = data.reshape((256,256))
Image.fromarray(img).save("inverted_image.png")

```

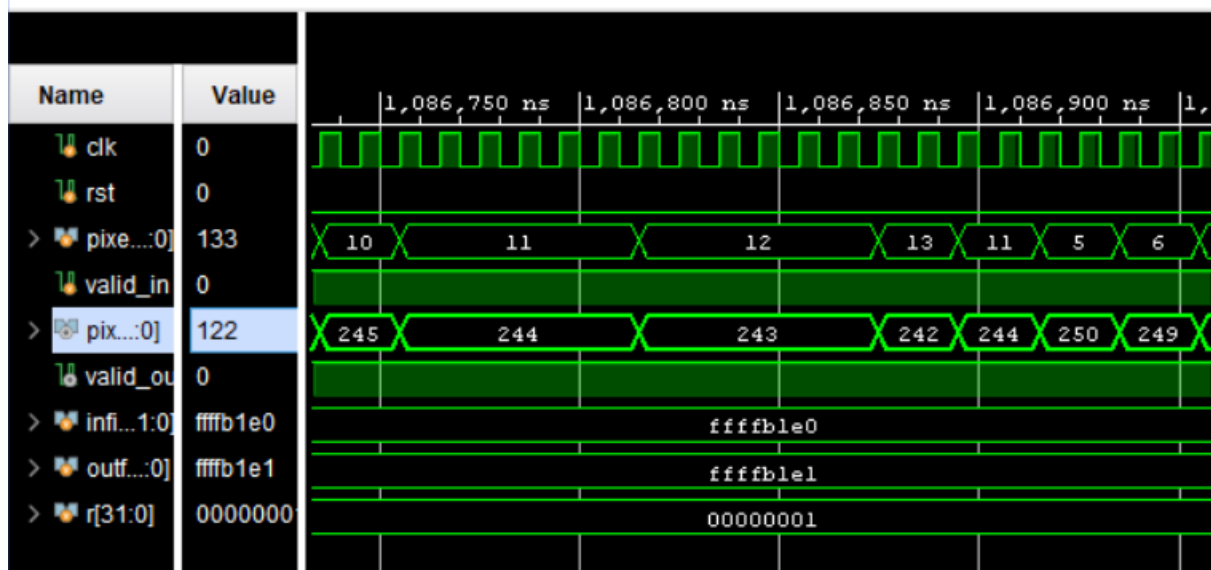
Output:



Grayscale image



Intensity inverted image



Result

The Verilog image intensity inversion system was successfully simulated. The 256×256 grayscale image was read from a text file and processed pixel-by-pixel by the UUT, where each pixel value was transformed using the intensity inversion operation. The output text file contained inverted grayscale values ranging from 0 to 255, confirming that each pixel was correctly converted according to the relation $\text{Output} = 255 - \text{Input}$. The reconstructed output image clearly appeared as the negative of the original image, with dark regions becoming bright and bright regions becoming dark, verifying the successful implementation of image intensity inversion using Verilog.